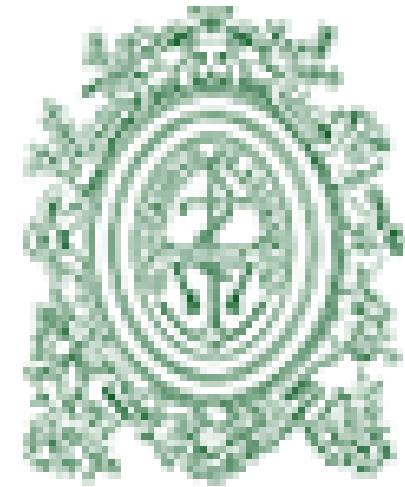




**UNIVERSIDAD
DE ANTIOQUIA**

Eliminación y Rebalanceo en Árboles B+

Manteniendo la Eficiencia y el Orden en Bases de Datos

Daniel Mesa Patiño
Alejandro Botero Arbeláez

Abril 2026

Estructuras de Datos y laboratorio

¿Qué es la Eliminación en un Árbol B+?

La **eliminación** es el proceso de remover una clave y su dato asociado del árbol, garantizando que el **árbol siga siendo válido** después de la operación.



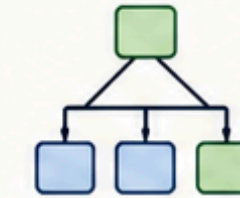
A diferencia de eliminar en una lista o un arreglo, en un árbol B+ **no basta con borrar el dato**. Hay que asegurarse de que el árbol mantenga tres cosas:

Lista / Arreglo



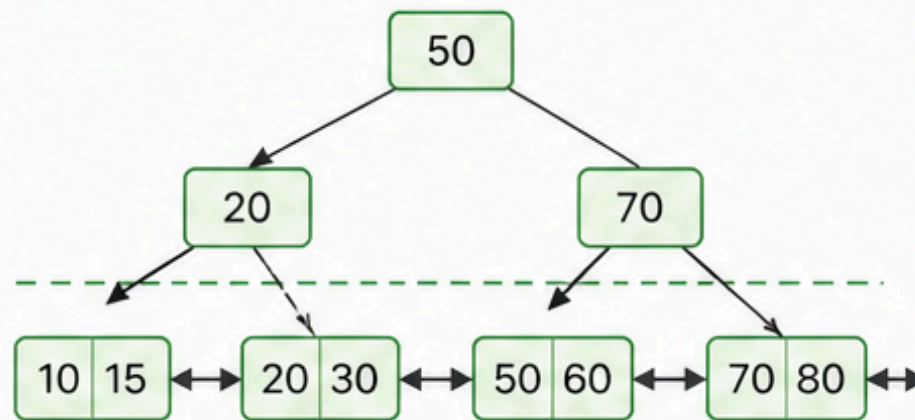
Solo se borra el dato.

Árbol B+



Hay que mantener la estructura válida.

1 Todas las hojas siguen a la misma altura

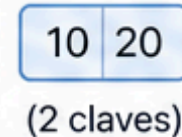


Después de eliminar, el árbol se ajusta para que todas las hojas queden en el mismo nivel.

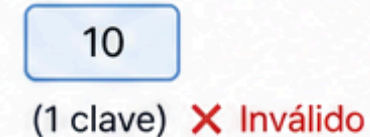
2 Cada nodo sigue teniendo el mínimo de claves permitido

Para un árbol de orden m , el mínimo es $\lceil m/2 \rceil - 1$ claves por nodo

Antes de eliminar

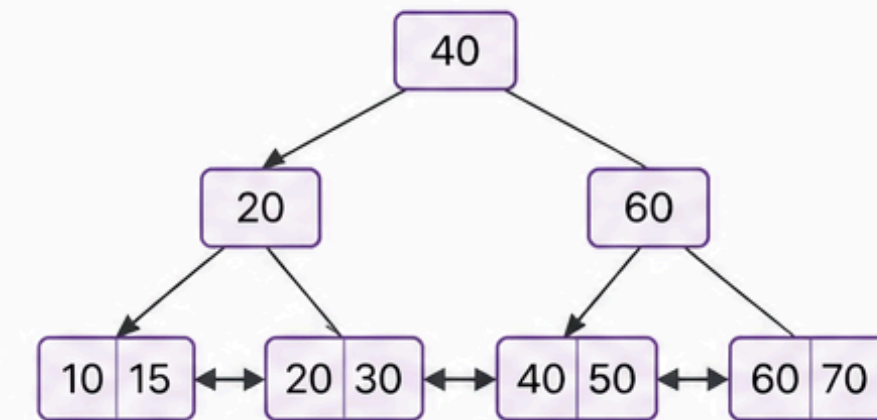


Después de eliminar



Se reestructura para volver a ser válido.

3 Las claves siguen ordenadas correctamente



El orden de las claves se mantiene en cada nodo y entre las hojas.

✓ Proceso de eliminación en un árbol B+



1 Buscar la clave en el árbol.



2 Eliminar la clave y su dato asociado.



3 Verificar si el árbol sigue siendo válido.



4 Si no es válido, reajustar (préstamo o fusión).



5 El árbol queda válido y balanceado.

Propiedades Clave de Árboles B+

La ocupación mínima es la regla que hace compleja la eliminación



Balanceo Perfecto:
todas las hojas a la misma altura



Hojas Conectadas:
punteros entre nodos hoja permiten recorrido secuencial



Datos en Hojas:
solo las hojas contienen datos reales



Búsqueda Rápida:
 $O(\log n)$



OCUPACIÓN VARIABLE — LA REGLA CRÍTICA
 $\lceil m/2 \rceil - 1 \leq n \leq m - 1$ claves por nodo
(excepto la raíz, que puede tener mínimo 1)

Si al eliminar un nodo queda con menos claves que este mínimo
→ subdesbordamiento → el árbol debe repararse



Inserción/Eliminación:
mantiene invariantes al rebalancear

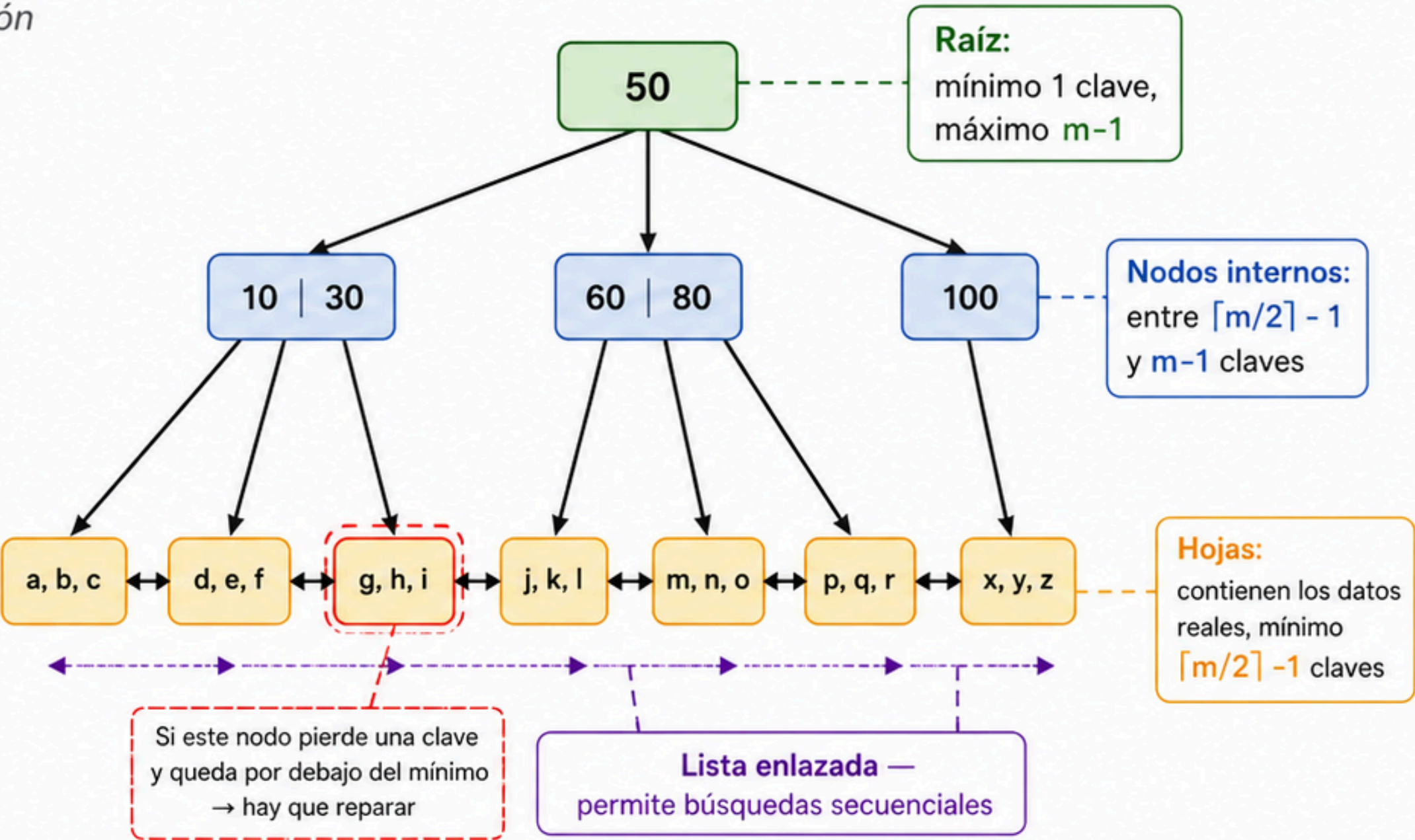


Resumen de mínimos y máximos según el orden (m)

Orden del árbol (m)	3	4	5	6
Mínimo de claves ($\lceil m/2 \rceil - 1$)	1	1	2	2
Máximo de claves (m - 1)	2	3	4	5

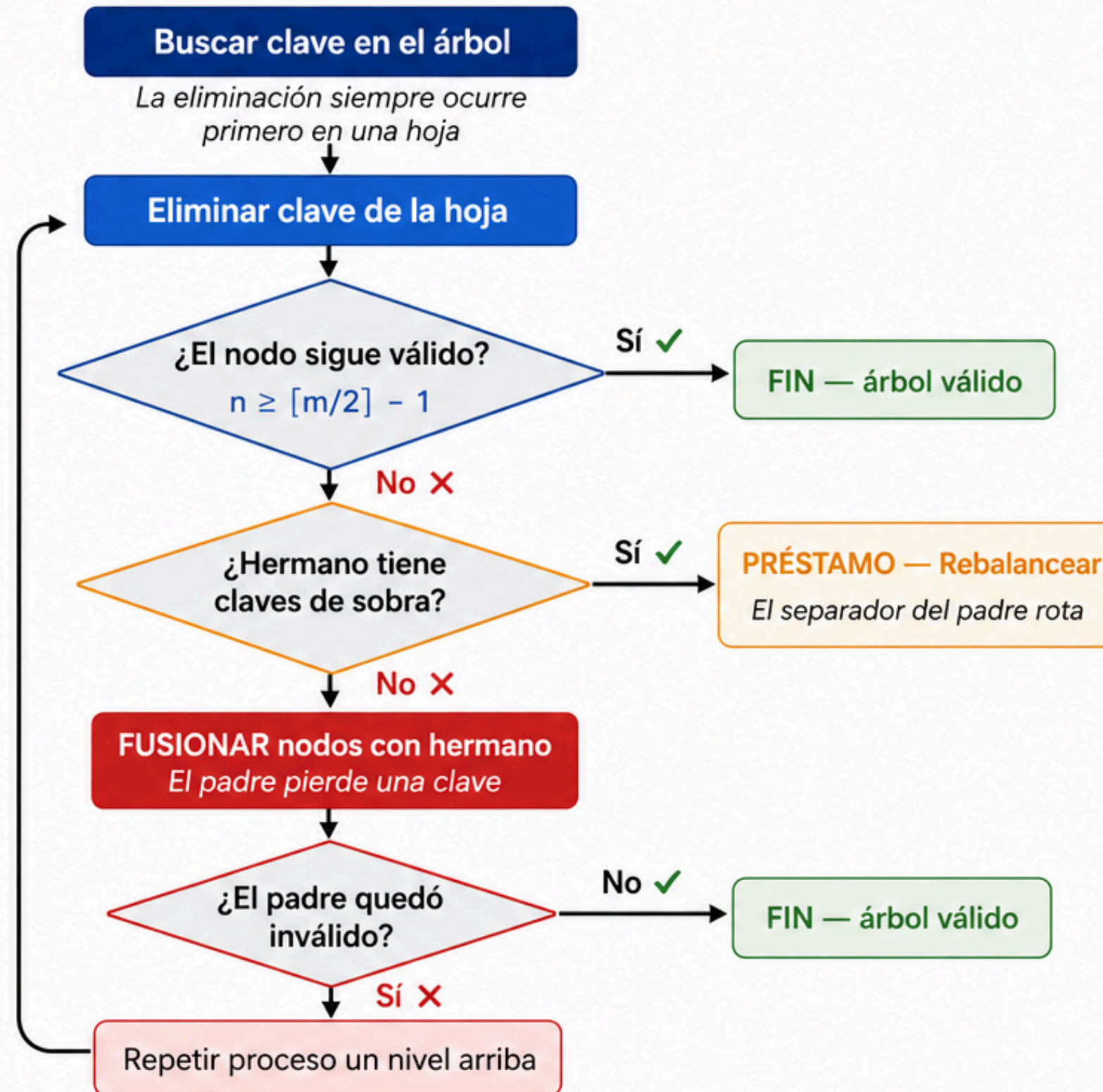


Fórmula:
mínimo = $\lceil m/2 \rceil - 1$
máximo = $m - 1$



Proceso General de Eliminación

Todo empieza en una hoja y puede propagarse hasta la raíz



Tres Caminos Posibles



✓ Sin Rebalanceo

- El nodo eliminado sigue teniendo $n \geq \lceil m/2 \rceil - 1$ claves
- **Resultado:** árbol válido inmediatamente
- **Complejidad:** $O(\log n)$ solo por la búsqueda



🔄 Préstamo (Borrowing)

- Un hermano tiene claves de sobra
- El separador del padre baja al nodo deficiente
- Una clave del hermano sube al padre
- **Resultado:** rebalanceo sin cascada
- **Complejidad:** $O(\log n)$



⌘ Fusión (Merging)

- Ningún hermano tiene claves de sobra
- Los nodos se combinan y el padre pierde una clave
- Puede propagarse nivel por nivel hasta la raíz
- **Complejidad:** $O(\log n)$ amortizado



La eliminación siempre ocurre en una hoja.

Nunca directamente en un nodo interno.

Esta es la diferencia clave entre el árbol B+ y el árbol B clásico.

Caso 1: Eliminación Simple

El caso donde el árbol no necesita ninguna reparación

- 1



Buscar la clave en el árbol
Bajamos por los nodos internos hasta encontrar la hoja que contiene la clave
- 2



Eliminar la clave directamente de la hoja
Se remueve la clave y su dato asociado
- 3



Verificar: ¿ $n \geq \lceil m/2 \rceil - 1$?
Sí ✓ → El nodo sigue cumpliendo el mínimo → FIN

Verificación del ejemplo

	Antes	Después
Nodo hoja	72, 90, 91 → 3 claves	72, 91 → 2 claves
¿Cumple mínimo?	✓ Sí	✓ Sí → FIN

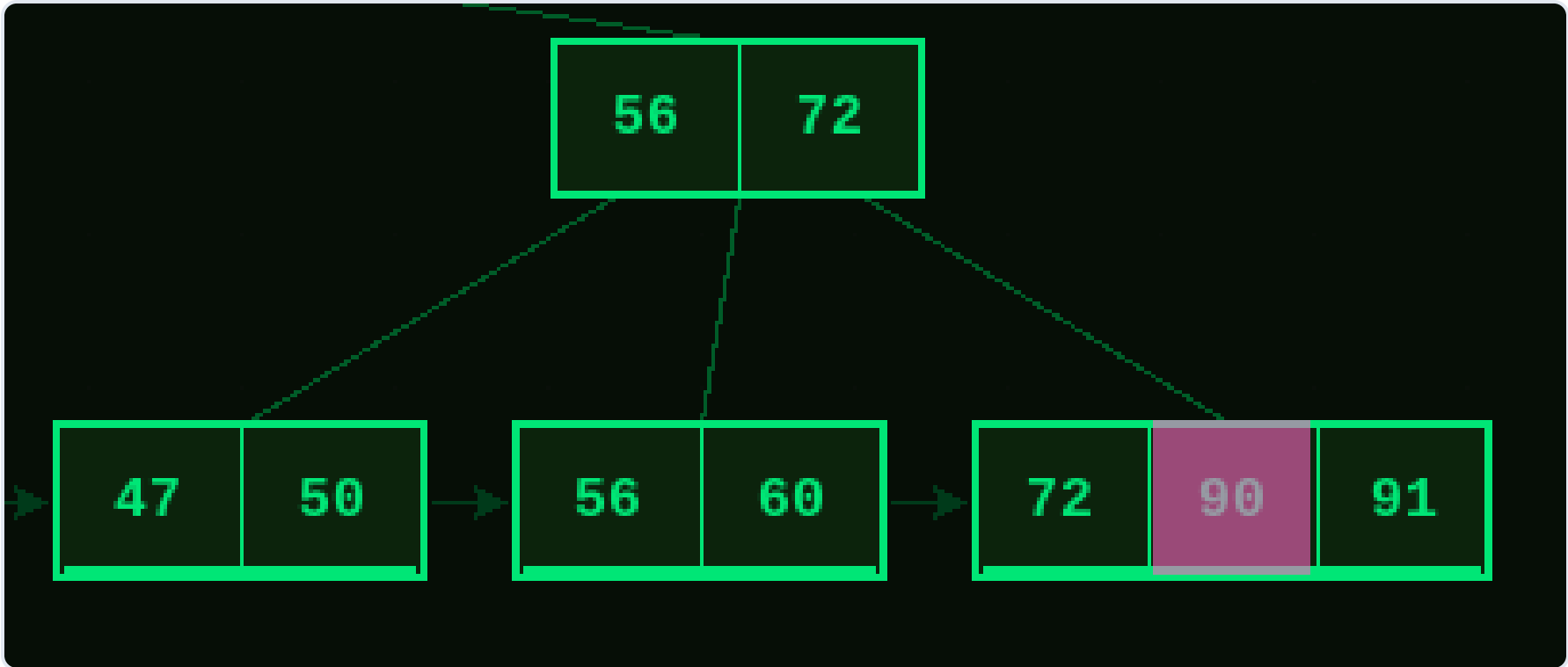
Árbol de orden 4 → mínimo = $\lceil 4/2 \rceil - 1 = 1$ clave
El nodo pasó de 3 a 2 claves. Sigue siendo mayor o igual a 1. Árbol válido.

 **El nodo sigue cumpliendo el mínimo**
No se necesita ninguna reparación. El árbol queda válido inmediatamente después del borrado.

 **Complejidad: $O(\log n)$**
Solo cuesta el tiempo de la búsqueda inicial.
No hay cascada, no hay movimiento de claves entre nodos, no hay rotación, no hay fusión.

 Este es el caso más común en la práctica cuando los nodos tienen buena ocupación. Es el caso ideal: rápido, simple y sin efectos secundarios.

Antes y Después



Caso 2: Préstamo (Borrowing)

Cuando el nodo queda inválido pero un hermano tiene claves de sobra

1



Detectar el subdesbordamiento

Después de eliminar, el nodo quedó con menos de $\lceil m/2 \rceil - 1$ claves. El árbol está inválido y debe repararse.

2



Buscar hermano con claves de sobra

Primero miramos el hermano izquierdo. Si no tiene exceso, miramos el hermano derecho. Un hermano tiene exceso cuando tiene más del mínimo requerido.

3



Realizar la rotación a través del padre

Este es el paso clave: la clave NO se mueve directamente entre hermanos. Pasa por el padre.

- El separador del padre baja al nodo deficiente
- Una clave del hermano sube al padre como nuevo separador
- El nodo deficiente recupera su mínimo



Complejidad: $O(\log n)$ pero sin cascada

El problema se resuelve completamente en este nivel. El padre no pierde ninguna clave porque la que bajó fue reemplazada por la que subió. No hay propagación hacia arriba.



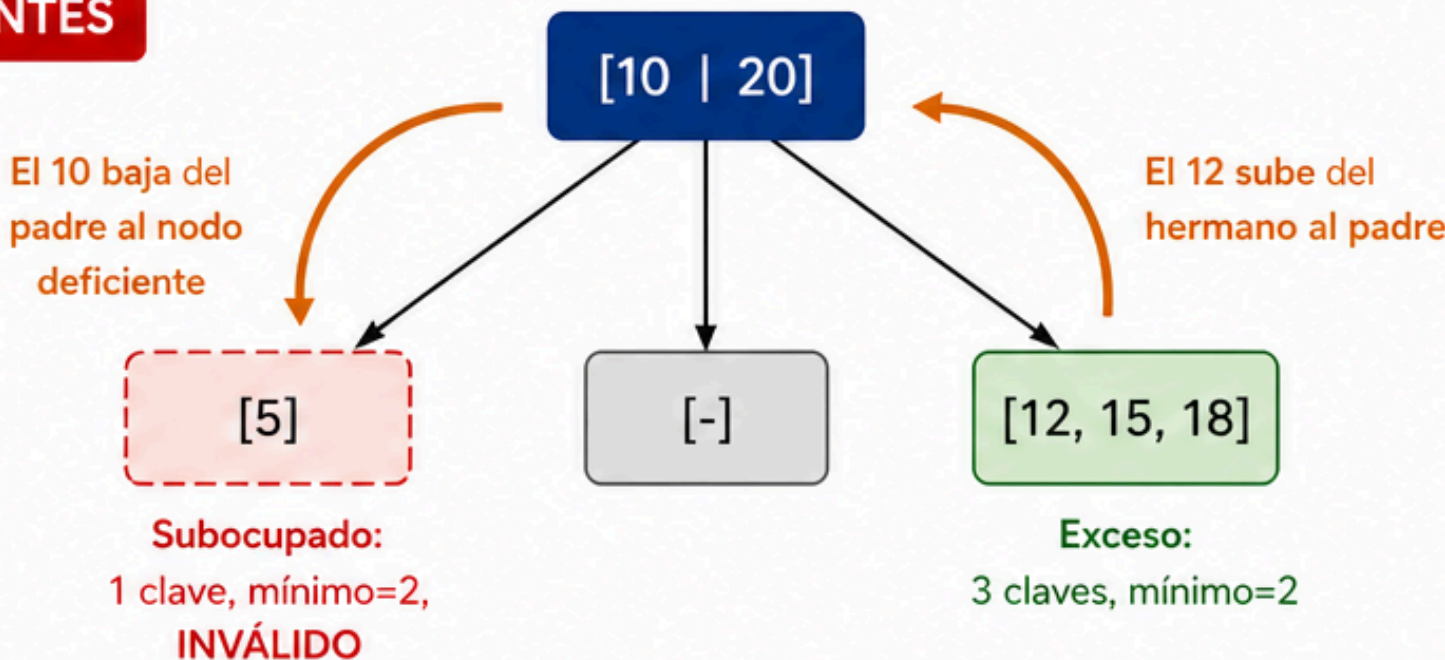
Orden de preferencia:

1. Intentar con hermano izquierdo primero
2. Si no tiene exceso, intentar con hermano derecho
3. Si ninguno tiene exceso → ir al **Caso 3: Fusión**

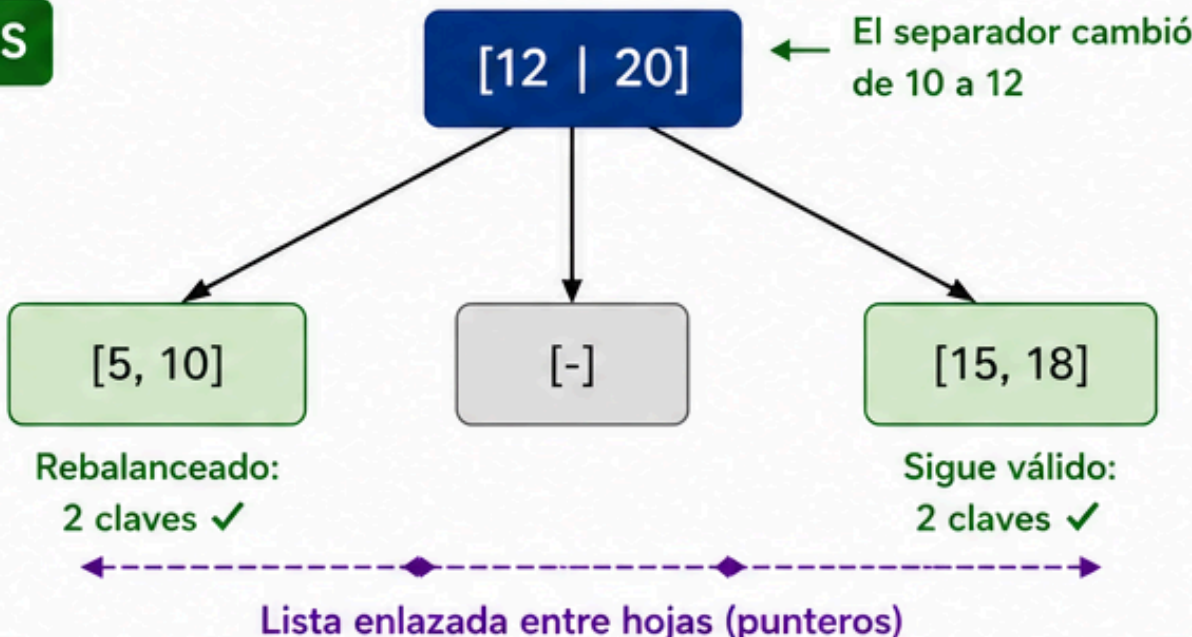


Idea clave: El préstamo mantiene el mismo número total de claves en el subárbol. Solo se redistribuyen para que todos cumplan el mínimo requerido.

ANTES



DESPUÉS



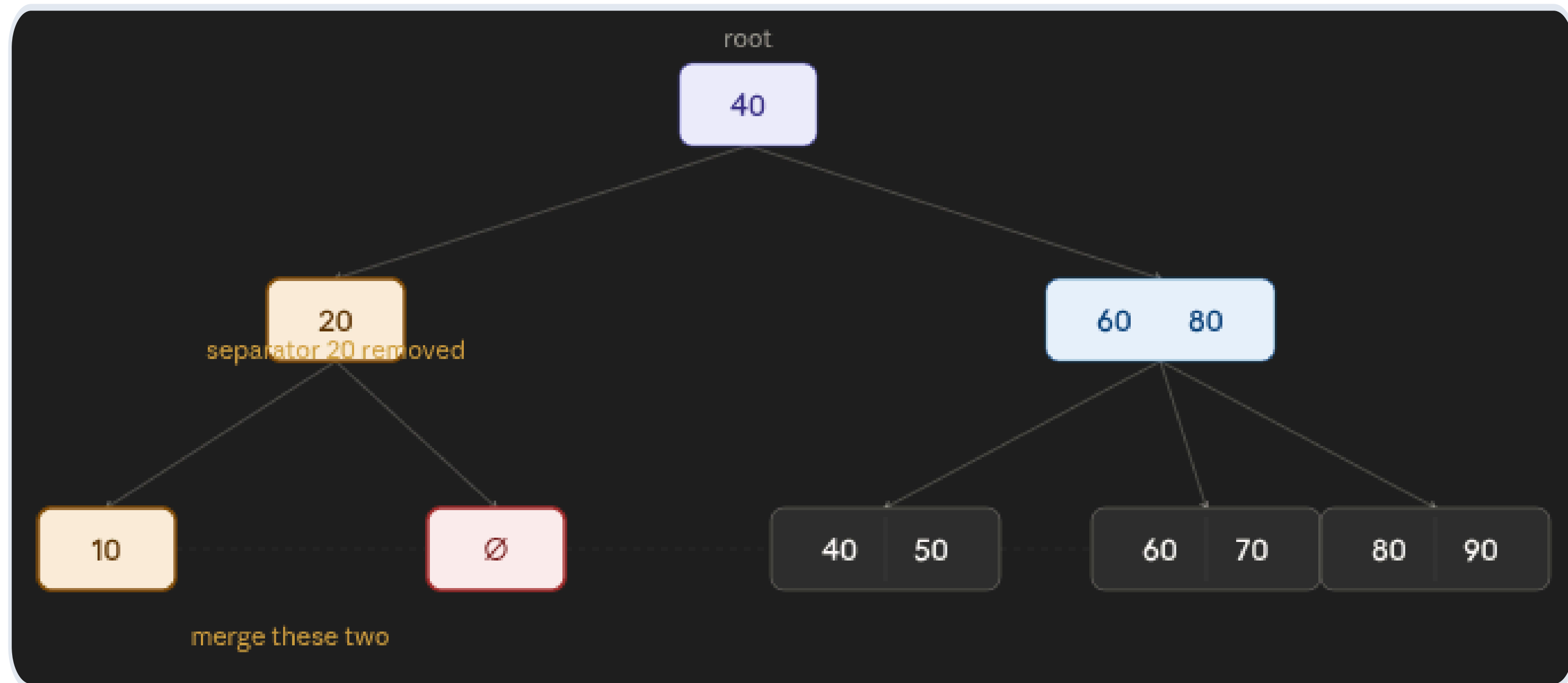
✓ Garantiza ocupación mínima

Después del préstamo, tanto el nodo deficiente como el hermano cumplen el mínimo de claves. El padre tampoco pierde claves. El árbol queda completamente válido sin necesidad de propagar el problema hacia arriba.

Nodo	Antes	Después	¿Válido?
Nodo deficiente	[5] → 1 clave	[5, 10] → 2 claves	✓ Sí
Hermano derecho	[12, 15, 18] → 3 claves	[15, 18] → 2 claves	✓ Sí
Padre	[10, 20]	[12, 20]	✓ Sin cambio de cantidad

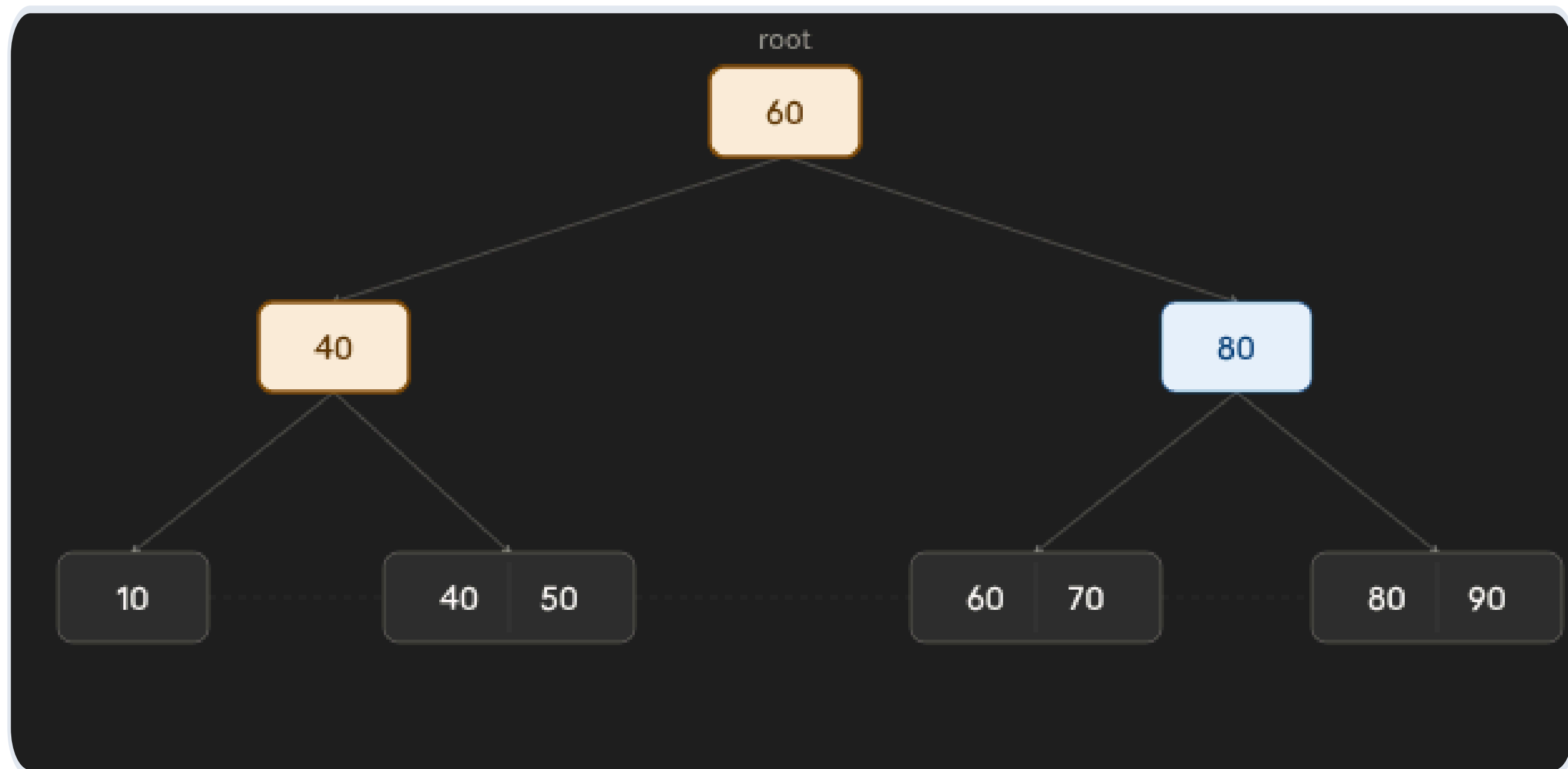
Caso 3: Fusión (Merging)

1. Nodo subocupado + hermano sin exceso
2. Separador cae al hijo, se fusionan
3. Padre pierde una clave (cascada posible)

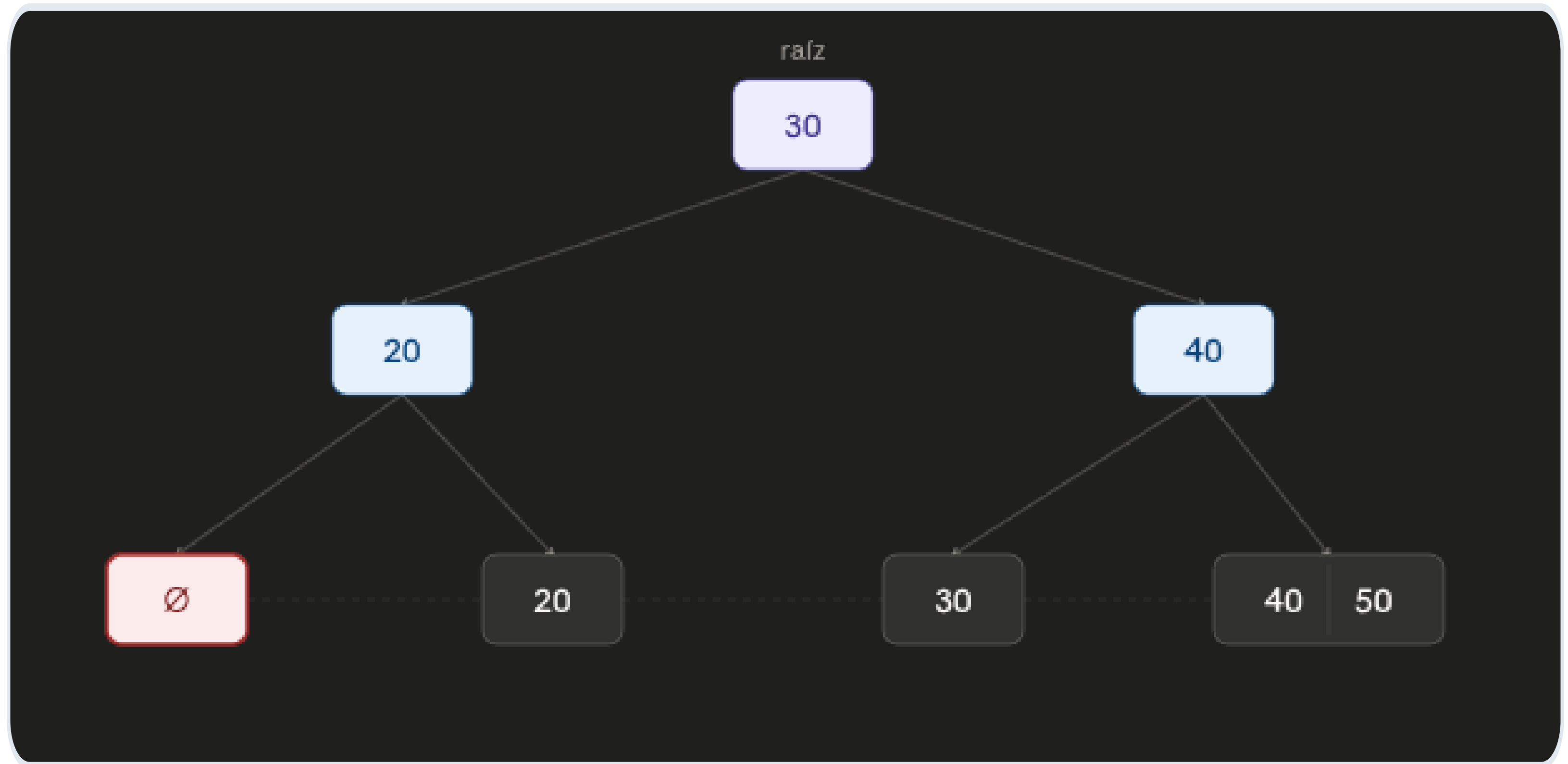


Caso 3: Fusión (Merging)

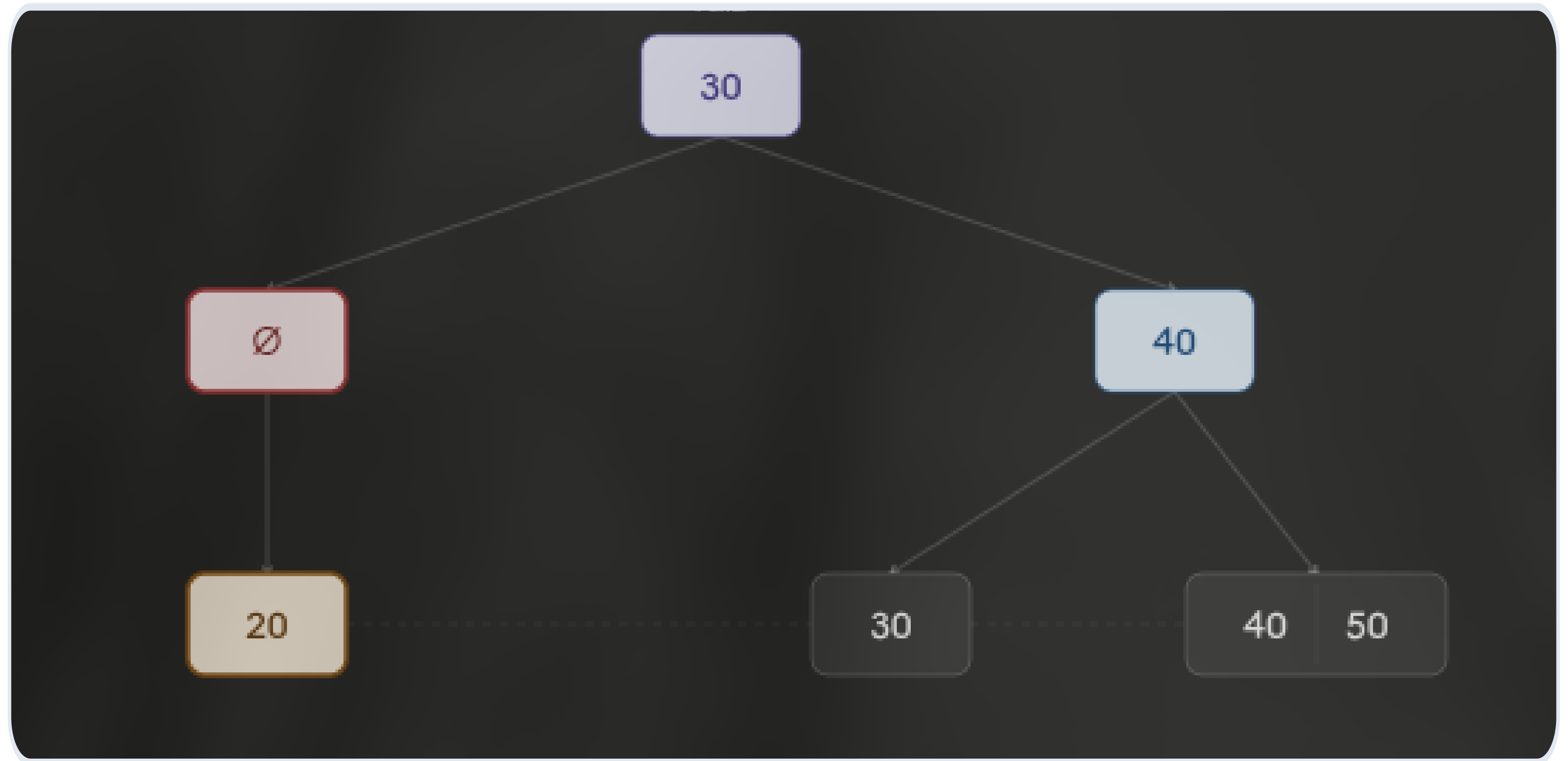
1. Nodo subocupado + hermano sin exceso
2. Separador cae al hijo, se fusionan
3. Padre pierde una clave (cascada posible)



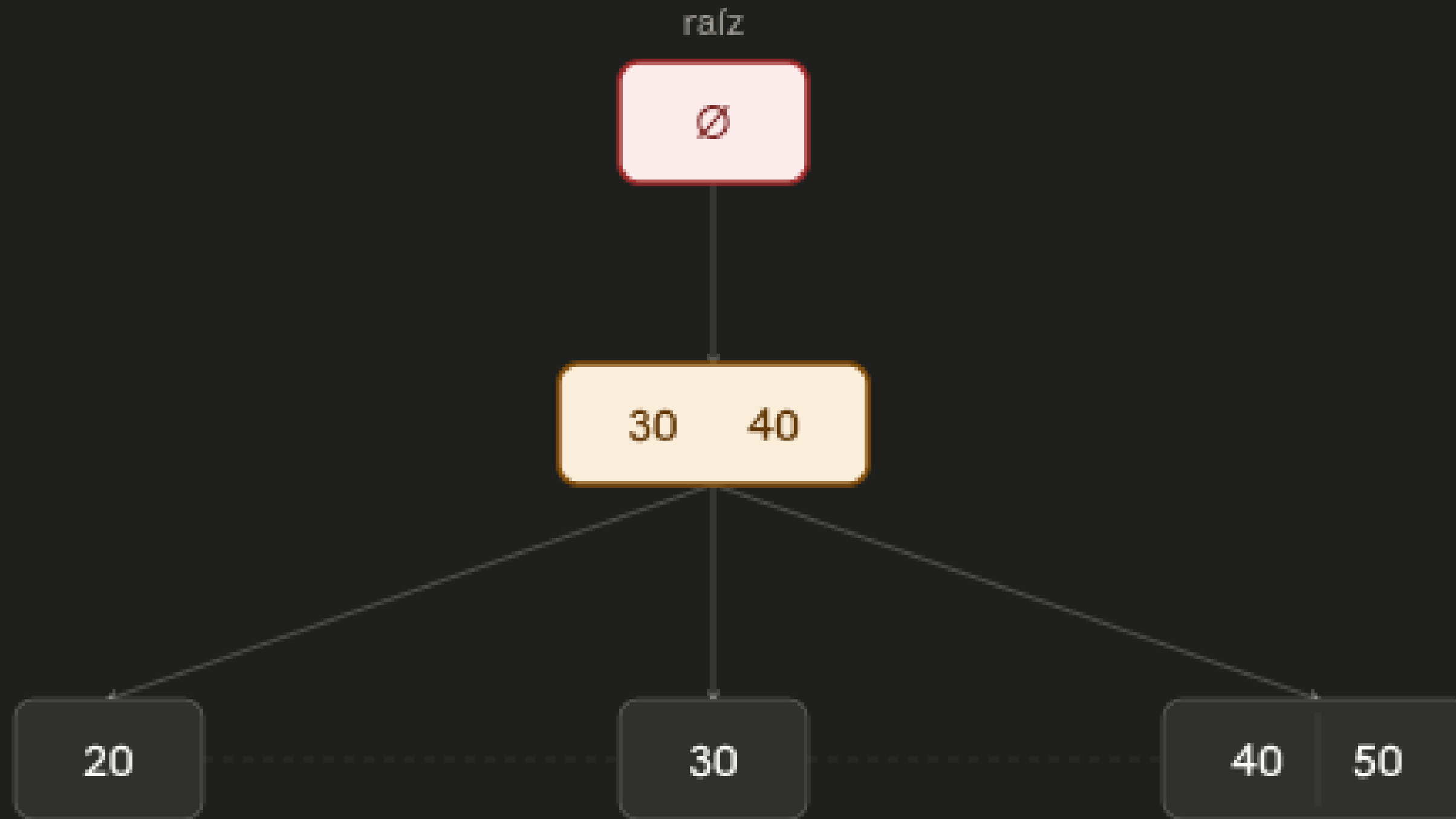
Propagación Hacia la Raíz



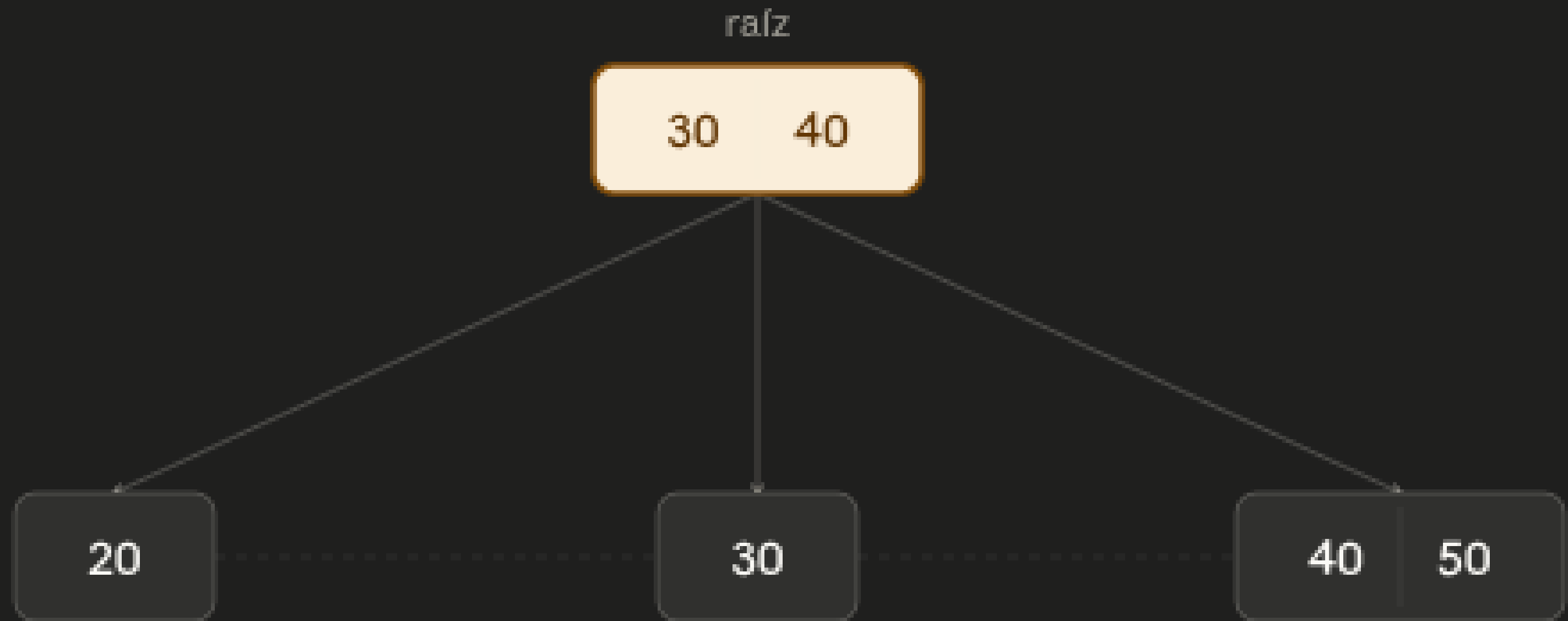
Propagación Hacia la Raíz



Propagación Hacia la Raíz



Propagación Hacia la Raíz



Complejidad Temporal

OPERACIÓN	CASO PROMEDIO	CASO PEOR	NOTAS
Búsqueda (punto)	$O(\log n)$	$O(\log n)$	Recorre desde la raíz hasta una hoja
Búsqueda por rango	$O(\log n + k)$	$O(\log n + k)$	k = número de elementos en el rango
Inserción	$O(\log n)$	$O(\log n)$	Posibles splits en cascada hasta la raíz
Eliminación	$O(\log n)$	$O(\log n)$	Posibles merges o redistribuciones
Acceso mínimo / máximo	$O(\log n)$	$O(\log n)$	Ir al nodo hoja más izquierdo/derecho
Recorrido ordenado	$O(n)$	$O(n)$	Lista enlazada de hojas permite scan lineal

n = número de claves almacenadas · orden del árbol t determina la constante oculta

Casos Especiales y Excepciones

CASOS EN NODOS HOJA			
CASO	ACCIÓN	CONDICIÓN	PROPAGACIÓN
1 Eliminación simple La hoja tiene más del mínimo de claves	Solo borrar	claves > $\lceil t/2 \rceil - 1$	Ninguna — árbol estable
2 Redistribución desde hermano izquierdo Underflow, pero hermano izquierdo tiene claves extra	Rotar derecha	Hermano izq. tiene > mínimo	Se actualiza la clave separadora en el padre
3 Redistribución desde hermano derecho Underflow, pero hermano derecho tiene claves extra	Rotar izquierda	Hermano der. tiene > mínimo	Se actualiza la clave separadora en el padre
4 Merge con hermano Underflow y ningún hermano puede prestar	Fusionar hojas	Ambos hermanos en mínimo	La clave separadora se elimina del padre → puede causar underflow en nivel superior

Casos Especiales y Excepciones

CASOS EN NODOS INTERNOS

CASO	ACCIÓN	CONDICIÓN	PROPAGACIÓN
5 Redistribución interna Nodo interno en underflow, hermano puede prestar	Rotar por padre	Hermano tiene > mínimo de hijos	Se rota una clave del padre hacia abajo; una del hermano sube al padre
6 Merge interno Nodo interno en underflow, ningún hermano puede prestar	Fusionar nodos	Ambos hermanos en mínimo	La clave separadora del padre baja al nodo fusionado → underflow sube al nivel siguiente
7 Colapso de la raíz La raíz queda con un solo hijo tras un merge	Eliminar raíz	Raíz tiene solo 1 hijo	El hijo único se convierte en la nueva raíz → altura del árbol disminuye en 1

Ventajas y Aplicaciones Reales

✓ Ventajas

Altura $O(\log n)$ para millones de claves

Múltiples claves por nodo = menos accesos a disco

Hojas enlazadas = escaneo secuencial eficiente

Perfectamente balanceado = garantías de peor caso

Rebalanceo automático = transparente al usuario

✗ Desventajas

Código rebalanceo más complejo que AVL

Cascadas pueden afectar múltiples niveles

Peor que hash para búsquedas puntuales puras

Sistemas Reales que Usan B+

NTFS

ext4

MySQL

PostgreSQL

SQLite

HDFS

MongoDB

Cassandra

Clave: Múltiples claves por nodo = enormemente menos accesos a disco comparado con árboles binarios

Referencias

Libro:

Ramakrishnan, R., & Gehrke, J. (2003). Sistemas de gestión de bases de datos (3.^a ed.). McGraw-Hill. <https://es.scribd.com/document/314037665/Sistemas-de-Gestion-de-Bases-de-Datos-3ra-Edicion-Ramakrishnam-Gehrke#page=1>

Página web:

**Programiz. (s.f.). Deletion from a B+ Tree.
de <https://www.programiz.com/dsa/deletion-from-a-b-plus-tree>**

Gracias por su atención